Docstring Generation for Python

https://gemini.google.com/app/f153f6d5054ce55b

User prompt: task_manager.py: ```python from datetime import date def add_task(tasks, title, due_date=None, priority="medium"): """Add a new task to the task list. Args: tasks (list): The list of existing task dictionaries. title (str): The task's title. due_date (date, optional): The task's due date. Defaults to None. priority (str): One of "high", "medium", "low". Defaults to "medium". Returns: list: The updated list of tasks. """ tasks.append({"title": title, "due_date": due_date, "priority": priority, "completed": False}) return tasks def mark_complete(tasks, title): """Mark a task as completed by title. Args: tasks (list): The list of task dictionaries. title (str): The title of the task to mark complete. Returns: bool: True if a matching task was found and updated, False otherwise. """ for task in tasks: if task["title"] == title: task["completed"] = True return True return False def get_overdue_tasks(tasks, current_date): overdue = [] for task in tasks: if task["due_date"] < current_date and not task["completed"]: overdue.append(task) return overdue def calculate_completion_rate(tasks): if not tasks: return 0.0 completed = sum(1 for t in tasks if t["completed"]) return completed / len(tasks) ``` Use the two provided docstrings of add_task and mark_complete as examples of the documentation style. Write a matching Google-style docstring for calculate_completion_rate, with a one-line summary, Args: and Returns:. Describe the actual return value and the empty-list behavior. Do not change the function logic. Return only calculate_completion_rate with its docstring in one Python code block.

Response:
```
def calculate_completion_rate(tasks):
    """Calculate the proportion of completed tasks in the task list.

    Args:
        tasks (list): The list of task dictionaries.

    Returns:
        float: The ratio of completed tasks to total tasks as a float between
            0.0 and 1.0, or 0.0 if the task list is empty.
    """
    if not tasks:
        return 0.0
    completed = sum(1 for t in tasks if t["completed"])
    return completed / len(tasks)
```
